

Rishi Fashion — SPA Project Report

CS3404: GUI Programming — Mini Project

1. Overview

Rishi Fashion is a Data-Driven Single Page Application for a batik saree e-commerce store, built with Vue 3 (Composition API), TypeScript, Vite, Tailwind CSS, and Pinia. It consumes live product data from the public DummyJSON API, using the womens-dresses, womens-bags, womens-jewellery, and womens-shoes categories as the product catalogue.

2. Features Implemented

- Product listing fetched from DummyJSON's /products endpoint, rendered in a responsive grid.
- Category filter (/products/category/:slug) and free-text search (/products/search).
- Client-side price sorting (Featured / Low-to-High / High-to-Low).
- Product Detail view with dynamic routing (/product/:id), an image gallery, and tabs for Description, Additional Information, Reviews, and Shipping & Delivery.
- Related products rail on the Detail view, drawn from the same category.
- Authentication simulation (bonus) against DummyJSON's /auth/login endpoint, storing the returned JWT access token and reflecting Log In / Log Out state in the navigation bar.
- Shopping cart (bonus) implemented as a Pinia store, persisted to localStorage so it survives page reloads, with quantity controls and live subtotal/shipping/total calculations.
- Dynamic routing (bonus) via Vue Router across Home, Product Detail, Login, Cart, Checkout, and Payment views.
- Dark mode (bonus) toggle using Tailwind's dark: variant, applied at the document root.
- A full checkout and payment flow recreating the project's Figma design: delivery details, order summary with shipping charge, payment method selection, card details entry, and a payment-success confirmation state.

3. Component Hierarchy

The application avoids a single monolithic App.vue. UI is broken into reusable presentational components (NavBar, ProductCard, FilterBar, ProductGrid, SiteFooter) and route-level views (HomeView, ProductDetailView, LoginView, CartView, CheckoutView, PaymentView). Structure:

```
App.vue
├── NavBar.vue (search, cart badge, auth state, dark-mode toggle)
├── RouterView
│   ├── HomeView.vue
│   ├── FilterBar.vue
│   ├── ProductGrid.vue → ProductCard.vue (×N)
│   ├── ProductDetailView.vue → ProductCard.vue (related)
│   ├── LoginView.vue
│   ├── CartView.vue
│   ├── CheckoutView.vue
│   ├── PaymentView.vue
│   └── SiteFooter.vue
```

4. State Management & Data Flow

Typed data (Product, CategoryOption) flows down to components via props, and user actions flow up via strictly-typed emit events (e.g. add-to-cart). Cross-cutting state that many unrelated components need — the cart contents and the authenticated user's session — lives in two Pinia stores (src/stores/cart.ts and src/stores/auth.ts) instead of being drilled through many prop layers. Both stores persist their state to localStorage, satisfying the 'persists across page reloads' bonus requirement.

5. Type Safety

All API responses are described by strict interfaces in src/types/ (Product, ProductListResponse, LoginResponse, AuthUser, etc.) with no use of the `any` type anywhere in the codebase. The axios wrapper in src/services/api.ts is fully generic-typed (axios.get<T>), so every consumer of ProductService/AuthService receives compile-time-checked data.

6. Architectural Diagram — Data Flow

Layer	Responsibility
types/	Strict interfaces describing DummyJSON API shapes
services/api.ts	Typed axios calls (ProductService, AuthService)
stores/	Pinia global state: cart.ts, auth.ts (localStorage-backed)
views/	Route-level pages, orchestrate data fetching + store actions
components/	Reusable, presentational, prop-driven UI pieces